# SnapStore: Automatic Image Processing on AWS

**S3 • AWS Lambda • IAM • CloudWatch**

Practical working report
AWS Region: US East (N. Virginia) — us-east-1
Runtime: Python 3.12 | Architecture: x86_64
Status: Implementation in progress — trigger and end-to-end test still require confirmation

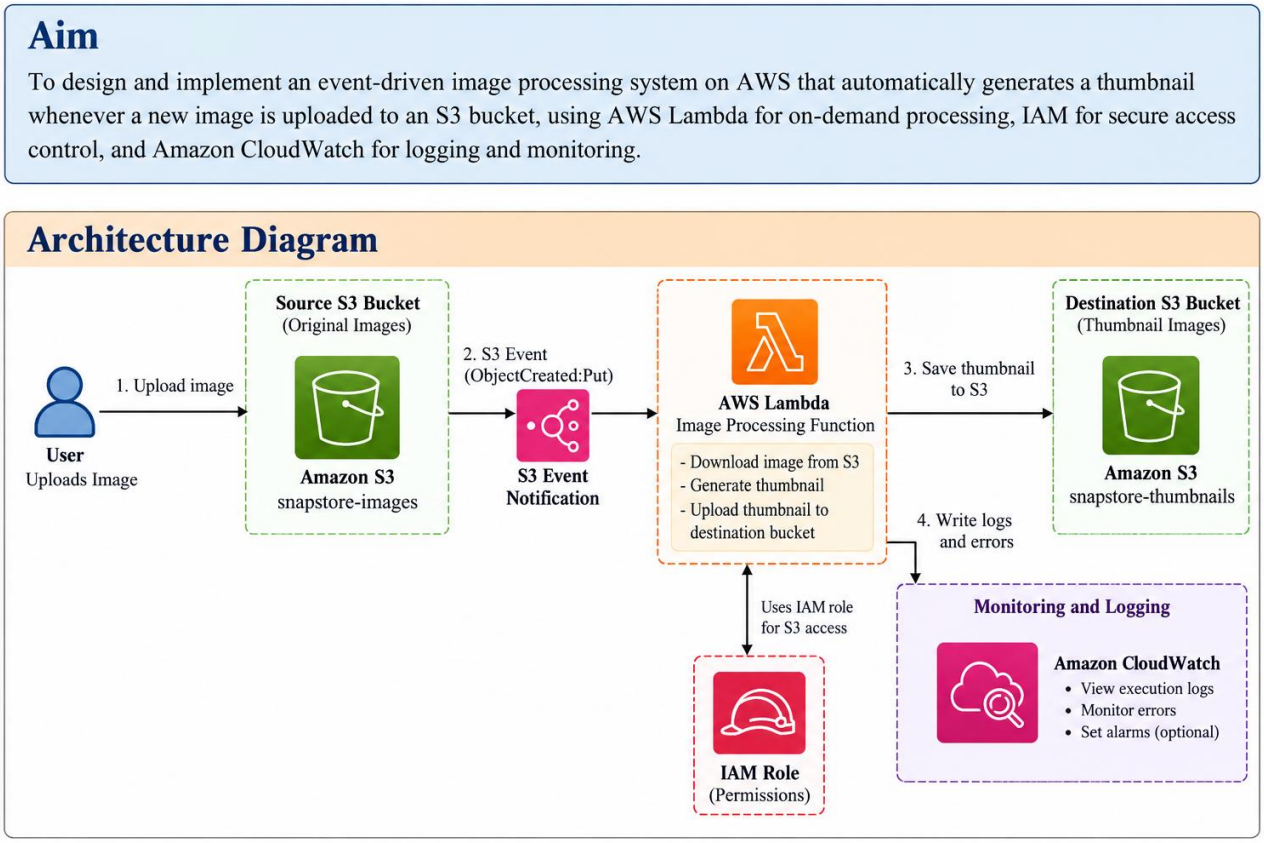
## 1. Aim

To design and implement an event-driven image-processing system on AWS that automatically generates a smaller thumbnail whenever a new photograph is uploaded to an Amazon S3 bucket, using AWS Lambda for on-demand processing, IAM for access control, and Amazon CloudWatch for execution logging and monitoring.

## 2. Problem Statement

SnapStore allows customers to upload photographs to Amazon S3. Each uploaded photograph needs a smaller thumbnail. Manual processing is time-consuming, so the system should process new objects automatically without a continuously running server. Administrators must be able to inspect execution logs and errors.

## 3. Architecture




Intended flow: user uploads an image to the source S3 bucket → S3 ObjectCreated event invokes Lambda → Lambda downloads and resizes the image using Pillow → thumbnail is written to the destination S3 bucket → Lambda execution logs are sent to CloudWatch. Lambda accesses AWS resources through its IAM execution role.

## 4. AWS Resources and Configuration

| Resource | Configuration | Purpose / Status |
|---|---|---|
| **Source S3 bucket** | snapstore-source-cct; us-east-1; public access blocked | Original uploaded images — created |
| **Destination S3 bucket** | snapstore-thumbnails-cct; us-east-1; public access blocked | Generated thumbnails — created |
| **IAM role** | SnapStore-Lambda-Execution-Role | Lambda execution role — created |

| | | |
|---|---|---|
| **IAM policies** | AmazonS3FullAccess; AWSLambdaBasicExecutionRole | S3 access and CloudWatch logging; broad S3 permission is acceptable for this short lab but should be narrowed for production |
| **Lambda function** | SnapStore-Image-Processor; Python 3.12; x86_64 | Image processing — created |
| **Lambda Layer** | SnapStore-Pillow, version 1; Python 3.12; x86_64 | Provides Pillow image library — published and attached |
| **S3 trigger** | Source bucket; All object create events; no prefix/suffix | Configuration page reached; successful creation not yet evidenced in screenshots |
| **CloudWatch** | Lambda log group /aws/lambda/SnapStore-Image-Processor | Execution logs/errors — to verify after test |

## 5. Procedure Performed

1. Opened Amazon S3 in the AWS Management Console and created the source bucket snapstore-source-cct in us-east-1.
2. Created the destination bucket snapstore-thumbnails-cct in the same region.
3. Opened IAM and created the role SnapStore-Lambda-Execution-Role with Lambda as the trusted service.
4. Attached AmazonS3FullAccess and AWSLambdaBasicExecutionRole to the execution role.
5. Created the Lambda function SnapStore-Image-Processor using Python 3.12 and x86_64 architecture, selecting the existing IAM role.
6. Opened AWS CloudShell and created a Python folder for the layer package.
7. Installed Pillow into the layer package directory using pip3 install Pillow -t python.
8. Published the layer as SnapStore-Pillow, version 1, compatible with Python 3.12 and x86_64.
9. Attached SnapStore-Pillow version 1 to the Lambda function.
10. Replaced the default Lambda handler with code that reads an S3 event, downloads the source image, creates a thumbnail with Pillow, and writes the result to snapstore-thumbnails-cct under a thumbnails/ key prefix.
11. Clicked Deploy and received AWS confirmation that the function was successfully updated.
12. Opened Add trigger and selected S3 as the intended event source. The screen showed All object create events selected; the source bucket still needed to be selected/confirmed and the trigger added.

## 6. Lambda Code Logic

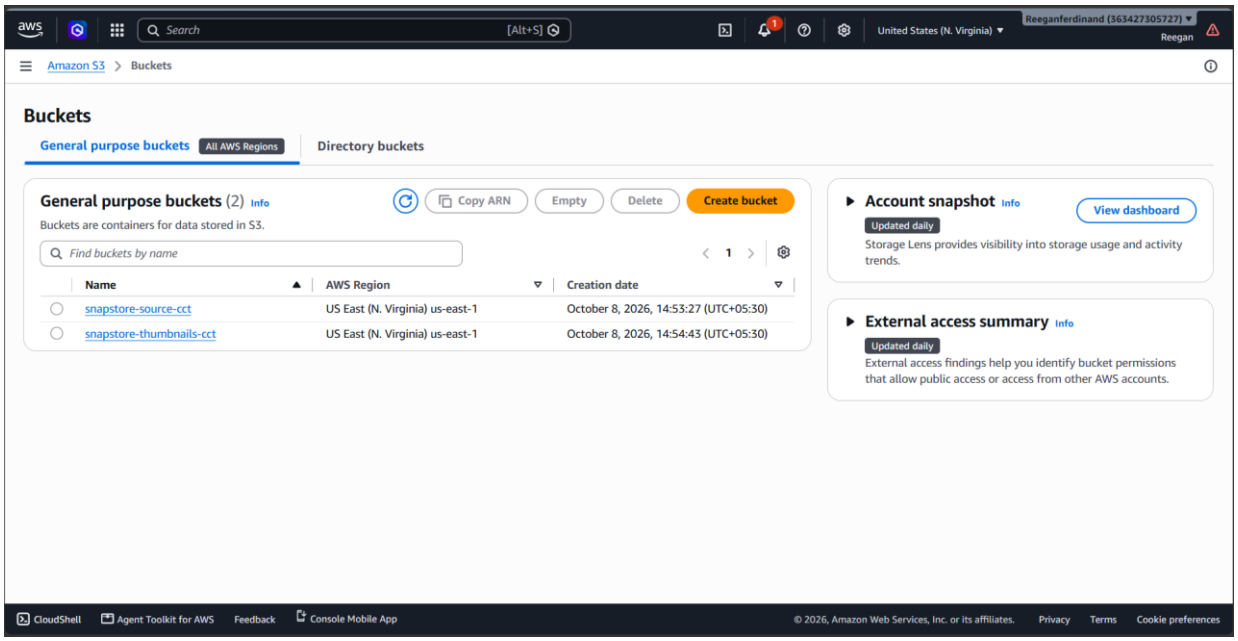
- Read the first record from the incoming S3 event.
- Extract the source bucket name and URL-decode the object key.
- Download the image using s3.get_object().
- Open the image in memory with Pillow and resize it to fit within 200 × 200 pixels.
- Write the resized image to an in-memory buffer.
- Upload the thumbnail to snapstore-thumbnails-cct using the key thumbnails/<original-object-key>.
- Print processing details so they can be inspected in CloudWatch Logs.

Important implementation note: the current handler assumes the uploaded object is a supported image and processes the first S3 record. The end-to-end test must confirm that the deployed code and Pillow layer work together. If testing reveals a problem, update the code and record the fix in the final report.

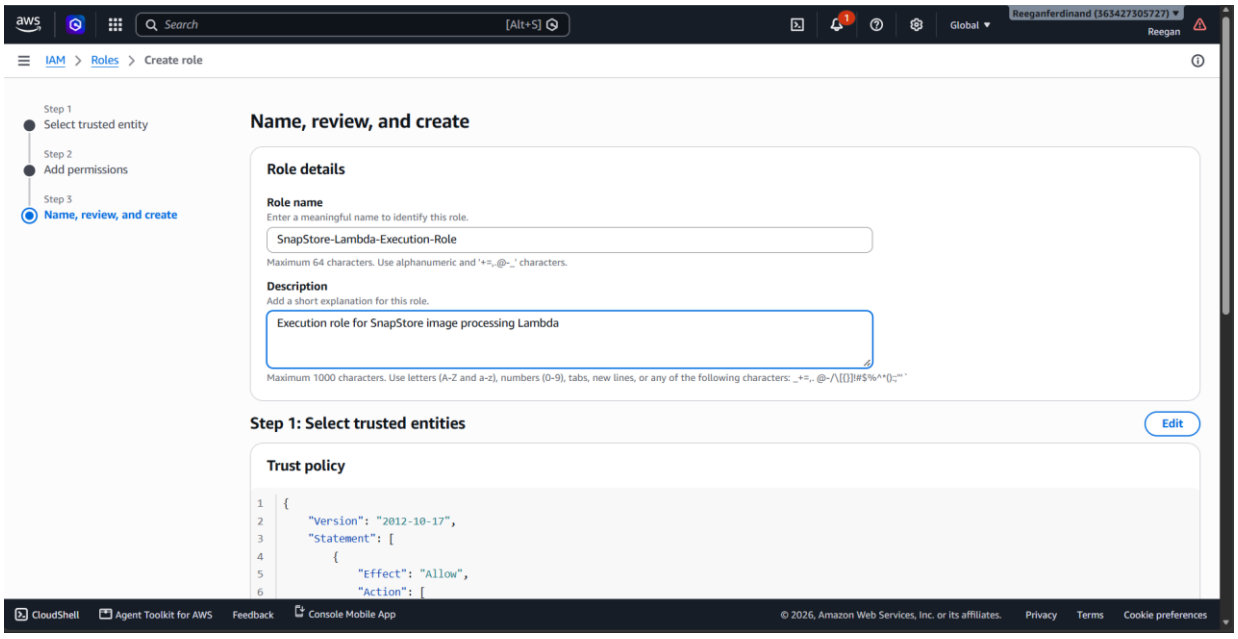
## 7. Evidence Screenshots

The following screenshots were supplied during the practical and are included as actual evidence. The final report can be updated with the successful trigger, test output, destination object, CloudWatch logs, and cleanup evidence.

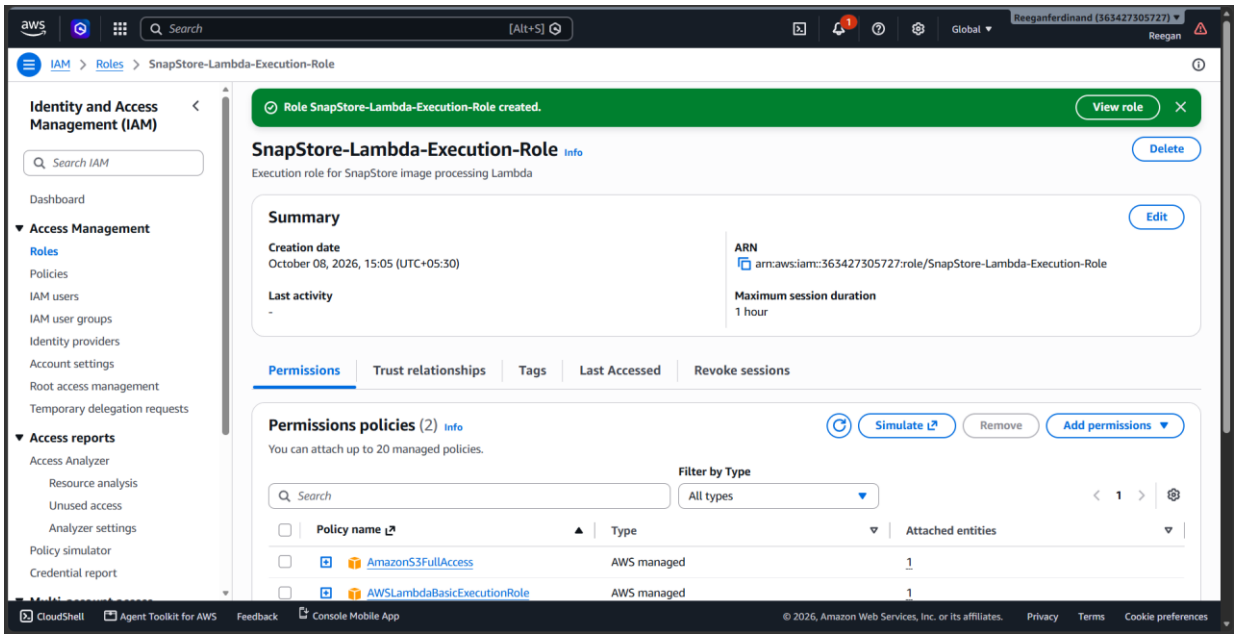
## Evidence 1 — Source and destination S3 buckets created in us-east-1.



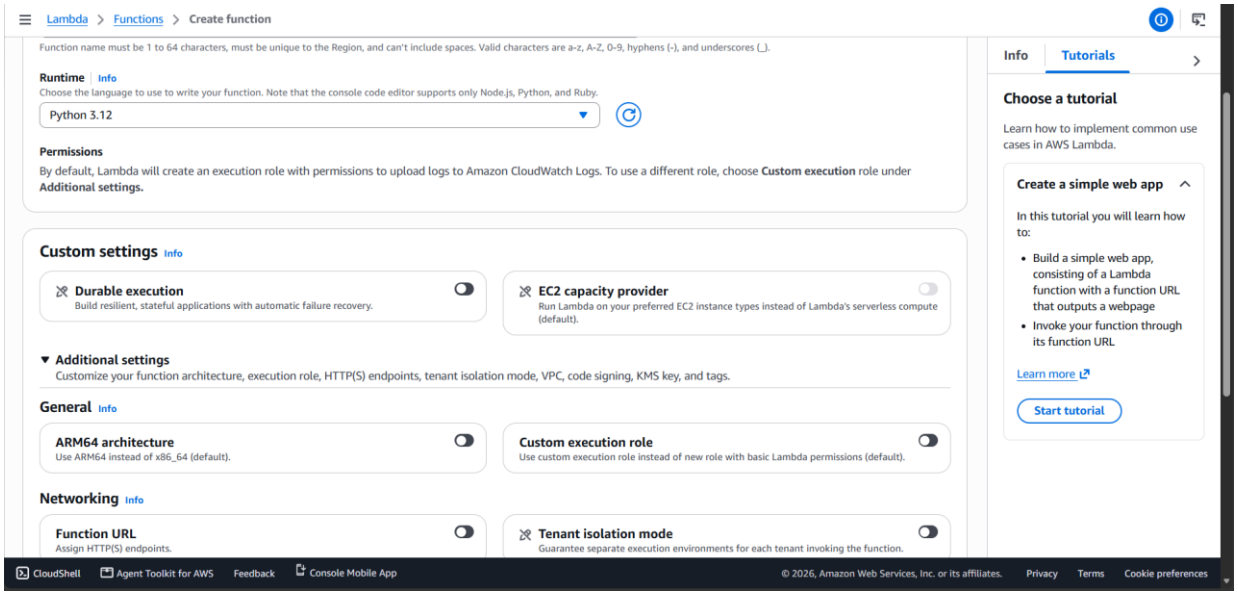

## Evidence 2A — IAM role creation details and trust policy screen.



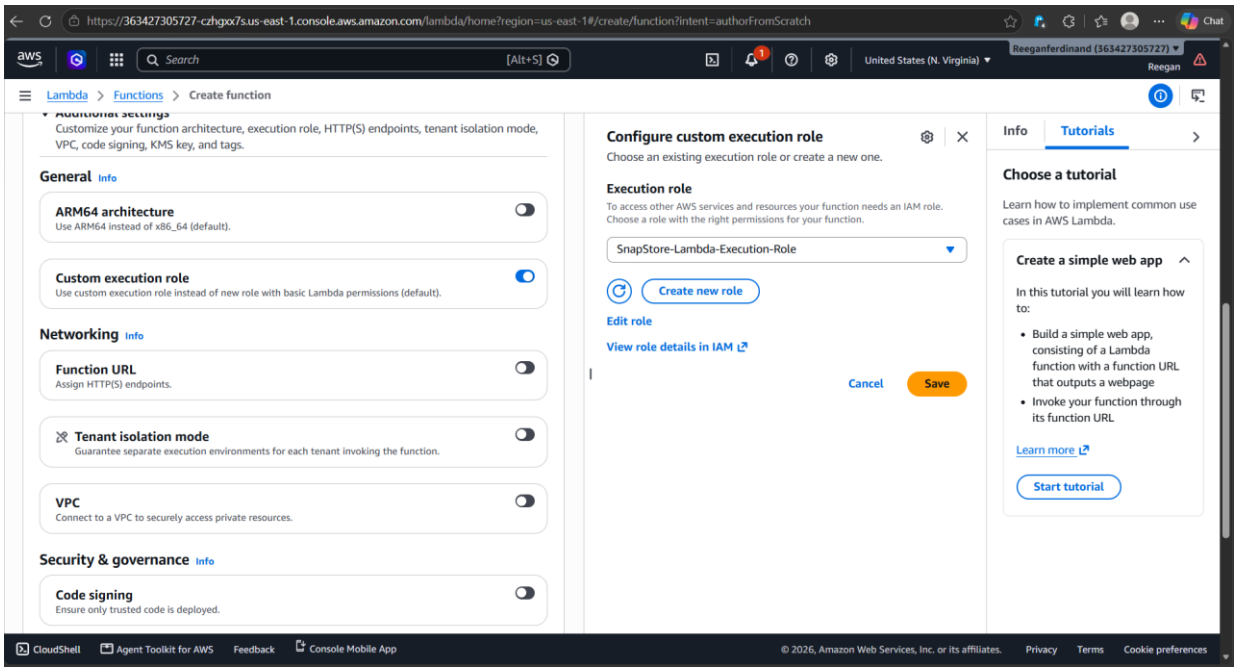
## Evidence 2B — IAM role created with the two attached policies.



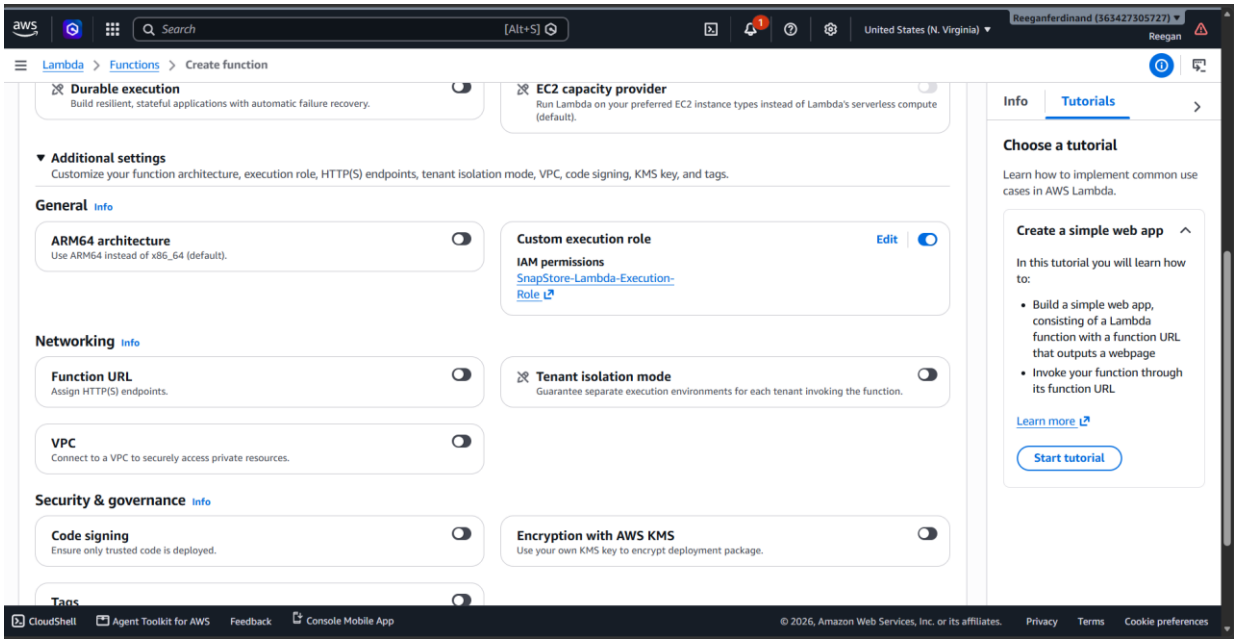

## Evidence 3A — Lambda create-function settings and custom execution role option.



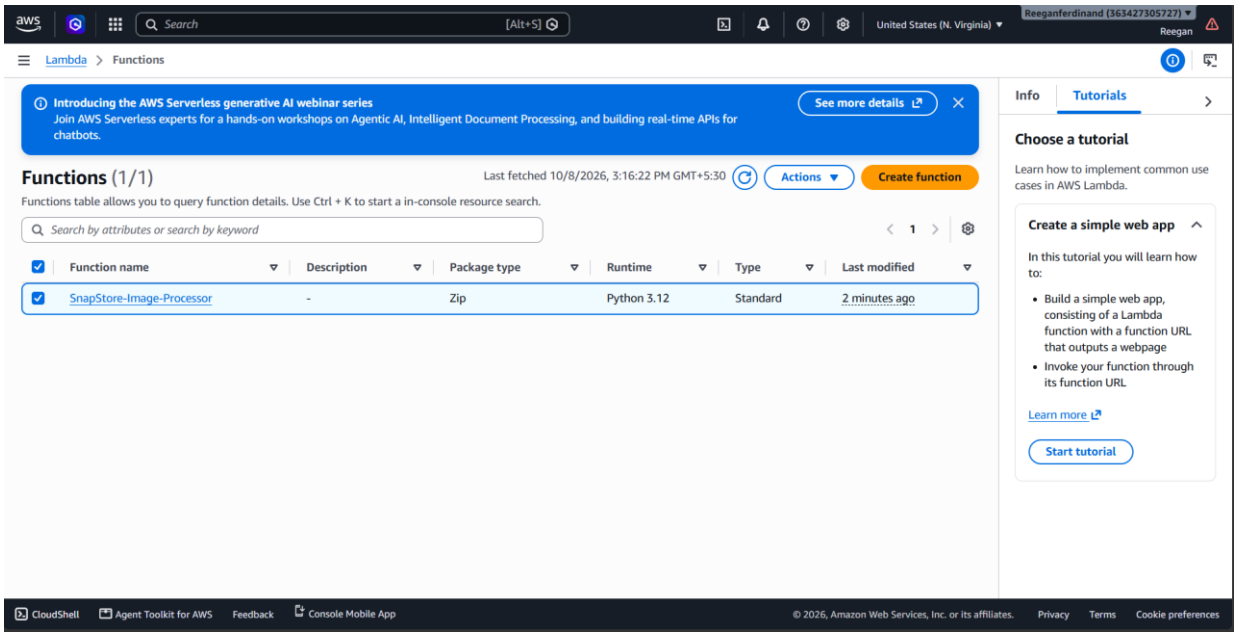
## Evidence 3B — Existing SnapStore IAM role selected.



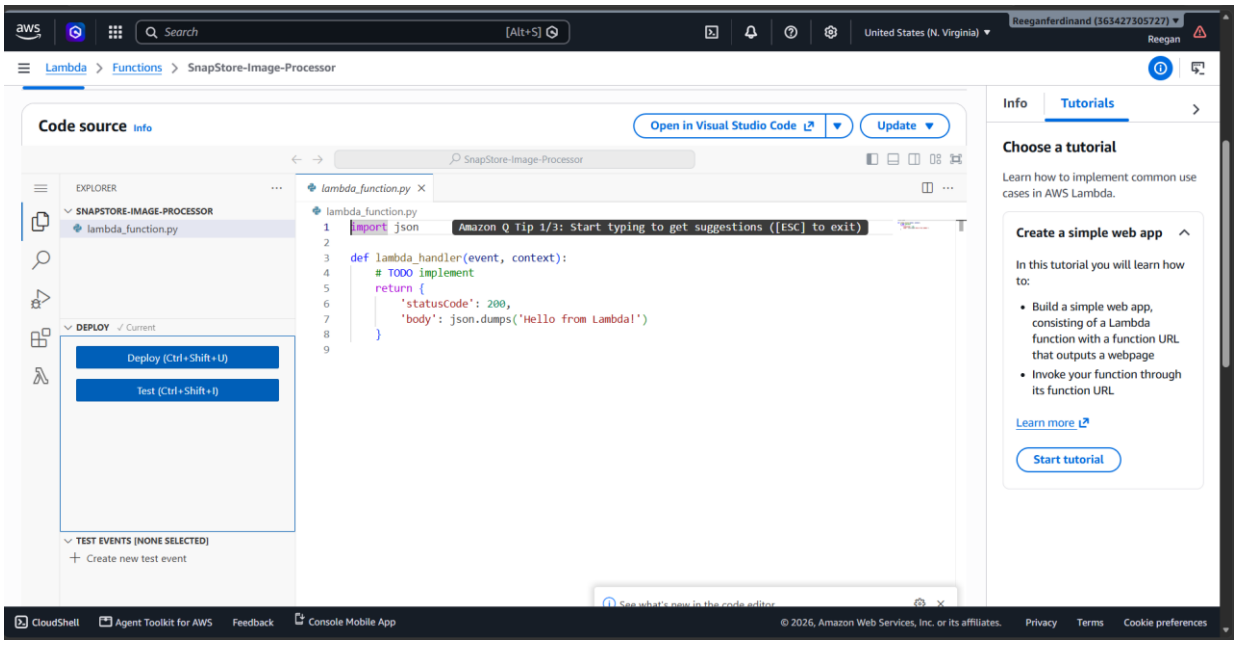

## Evidence 3C — Lambda custom execution role setting confirmed.



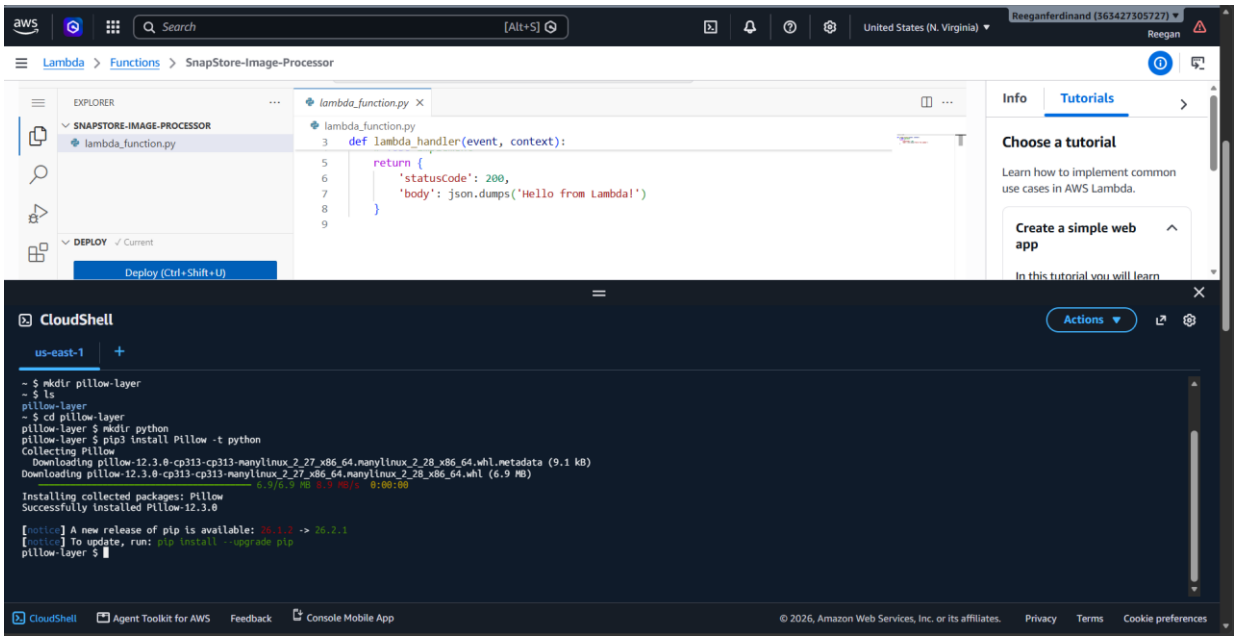
## Evidence 4 — SnapStore-Image-Processor created with Python 3.12.



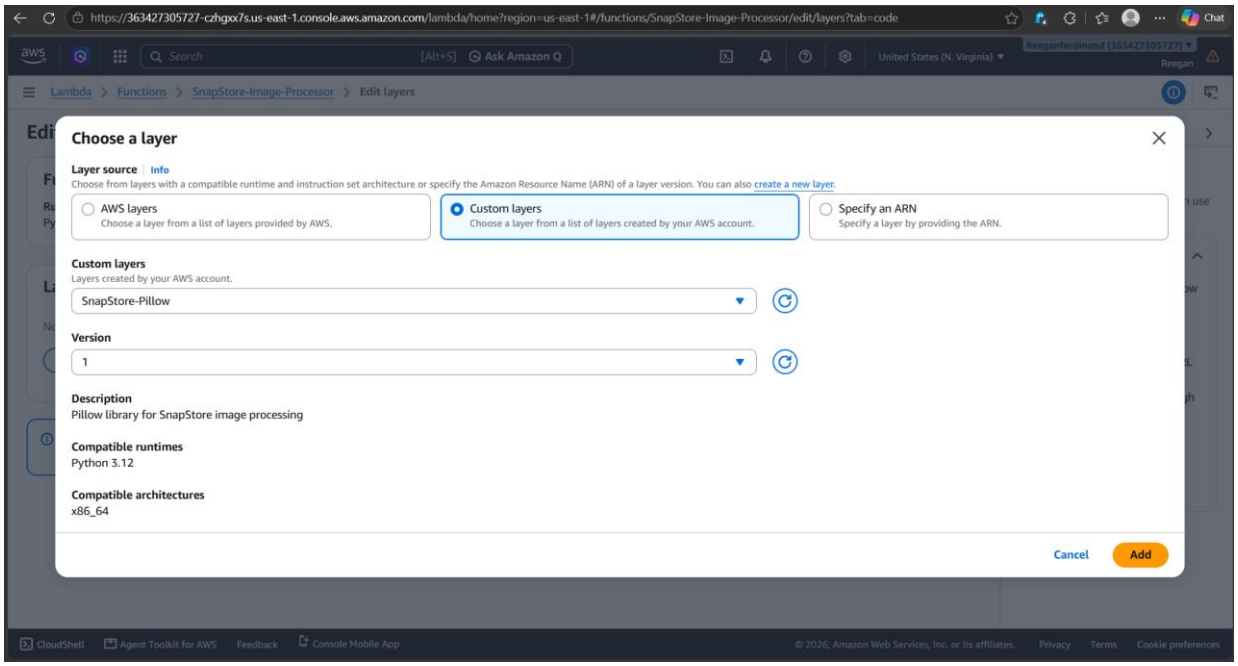

## Evidence 5 — Default Lambda code editor before implementation.



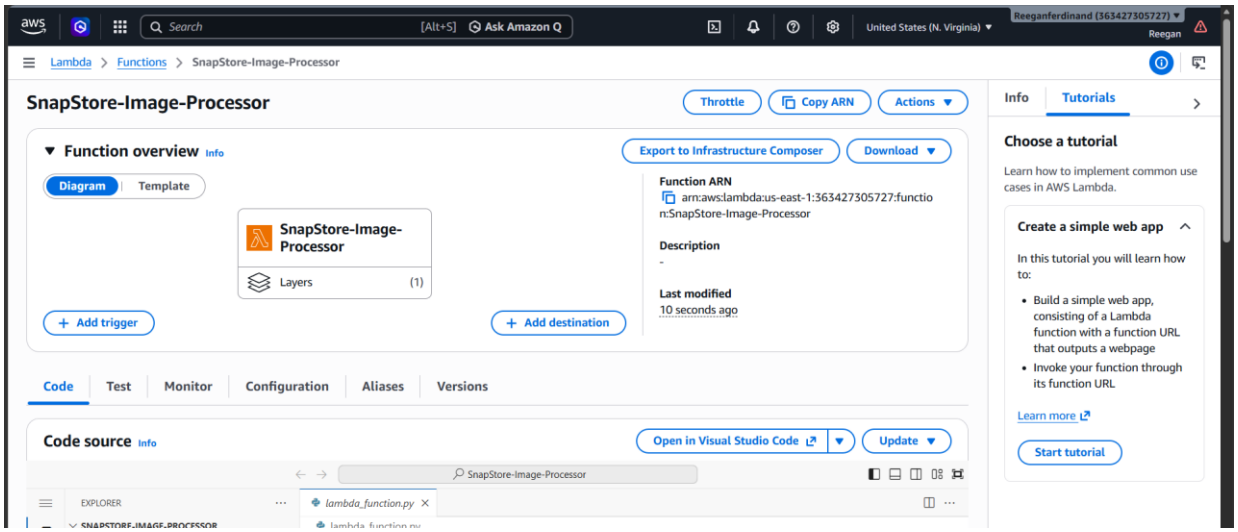
## Evidence 6A — CloudShell Pillow installation and layer publish command/output.



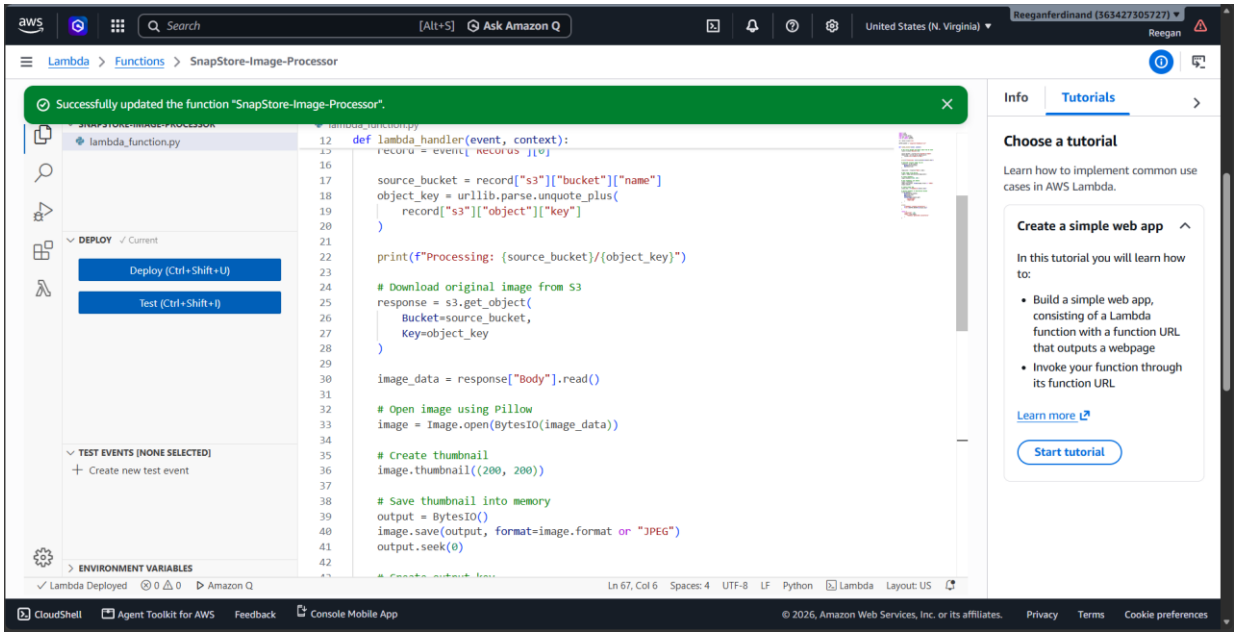

## Evidence 6B — Pillow custom layer selection, version 1.



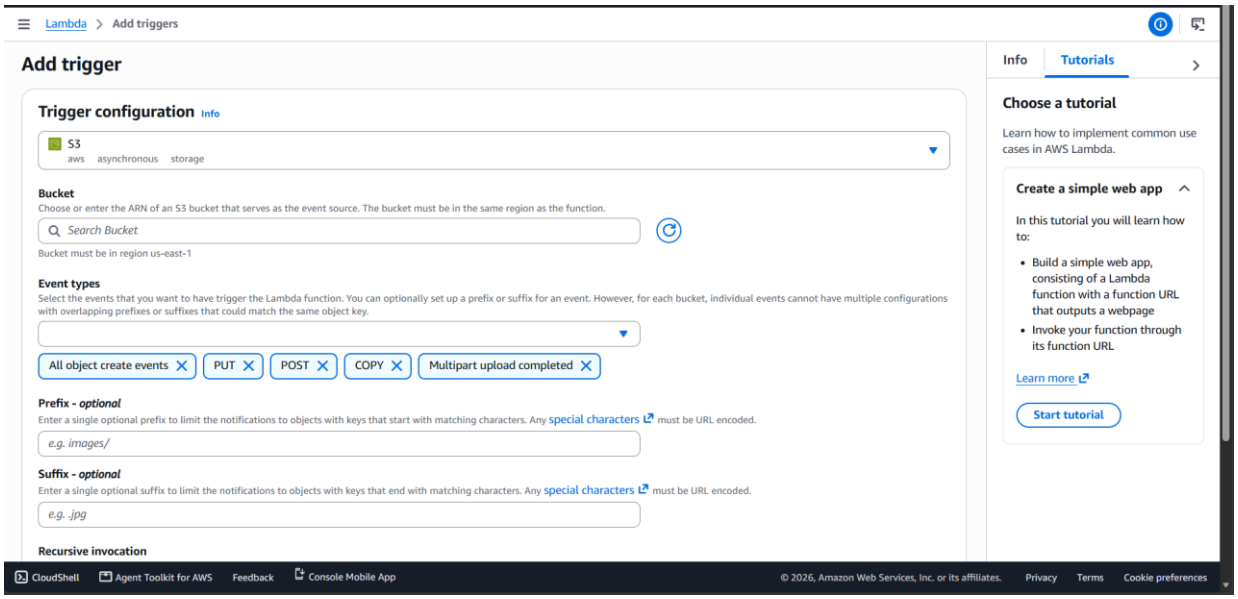
## Evidence 6C — Lambda overview showing one attached layer.




## Evidence 7 — Lambda image-processing code deployed successfully.



### Evidence 8 — S3 trigger configuration page; source bucket selection/addition not yet confirmed.




## 8. Testing and Expected Output

Testing status: pending. The trigger and end-to-end execution have not yet been evidenced.

Planned validation:

13. Add the S3 trigger to snapstore-source-cct for all object-create events.
14. Upload a small JPG or PNG file to the source bucket.
15. Confirm Lambda invocation in the Lambda Monitor tab or CloudWatch metrics.
16. Confirm a thumbnail object appears in snapstore-thumbnails-cct under thumbnails/.
17. Open CloudWatch Logs for /aws/lambda/SnapStore-Image-Processor and check for the processing and success messages.
18. If an error occurs, record the error and the fix before finalizing the report.

## 9. Security and Cost-Safety Notes

- Keep both buckets private; do not enable public access for this lab.
- The current AmazonS3FullAccess policy is broad. A least-privilege policy restricted to the source bucket's s3:GetObject and destination bucket's s3:PutObject should be used for production.
- Use only a few small test images and avoid unnecessary paid AWS services.
- Keep resources in us-east-1 to match the Lambda function and S3 trigger requirements.
- After capturing final evidence, delete lab resources if they are no longer needed. Remove the S3 trigger before deleting its source bucket if necessary; empty and delete buckets, delete the Lambda function and layer if no longer needed, and delete the IAM role after detaching policies.
- Check the AWS Billing/Free Tier dashboard for actual usage; free-tier eligibility and charges depend on account type, offer, and usage.

## 10. Result and Conclusion

The core AWS resources have been created: two S3 buckets, an IAM execution role, a Python 3.12 Lambda function, and a Pillow layer. The image-processing code has been deployed. The practical is not yet fully verified because the S3 trigger addition, thumbnail output, and CloudWatch execution logs still need to be confirmed. The final result statement should be updated after the end-to-end test succeeds.